

HANI MAAK — هاني معاك

FULL DEVELOPMENT-READY PRODUCT REQUIREMENTS DOCUMENT

Version 2.0 — Advanced Investor / Competition MVP

Prepared: 16 September 2026

Product category: Patient Journey Infrastructure

Business model: B2B White-Label SaaS

Primary implementation: Next.js mobile-first responsive PWA + staff web console + phone-based AI agent

Demonstration facility: Hôpital Charles Nicolle, Tunis (real institution; indoor prototype geometry is explicitly simulated until hospital validation)

DOCUMENT PURPOSE

This PRD is intended to be detailed enough that a full-stack engineering team — or ChatGPT in the next development session — can implement Hani Maak without re-interpreting the product from scratch. It defines the product behavior, data model, architecture, API and tool boundaries, AI behavior, safety rules, user experience, third-party integrations, evaluation plan, demo assumptions, known limitations, cost model, and evolution path.

This is not a generic booking app specification. Appointment scheduling is only the entry point. The product is designed around one continuous patient journey: ACCESS → GUIDANCE → CONTINUITY.

IMPORTANT PRODUCT BOUNDARY

Hani Maak may automate administrative and informational work, but it must not independently diagnose, prescribe, change a dosage, determine clinical urgency, or certify that a medication is safe for a specific patient. Clinical content must originate from an authorized provider or an approved knowledge source. When the system is uncertain or the request crosses a clinical boundary, it must escalate to a human.

0. DOCUMENT CONTROL

0.1 Status

- Status: Build-ready product specification for advanced competition MVP.
- Intended audience: product, engineering, UI/UX, AI/voice, healthcare operations, security/privacy, QA, investors and competition jury.
- Decision level: product direction is fixed; implementation details may be adjusted if technical testing proves a different implementation safer or more reliable without changing the product concept.

0.2 Product working name

Hani Maak — هاني معاك.

Meaning: “I’m with you / I’m here with you.” The name represents presence, guidance and continuity throughout a healthcare journey.

0.3 Design principles

P1 — Simplicity before feature density.

P2 — Mobile-first, but never smartphone-only.

P3 — A patient should always know “What is my next step?”

P4 — The AI speaks naturally but acts only through constrained, auditable tools.

P5 — Administrative automation may be autonomous inside configured rules; clinical judgment remains human.

P6 — The same backend state powers the mobile experience, staff console and phone agent.

P7 — Every high-impact write action is confirmed, validated and logged.

P8 — Demo realism must never be confused with validated clinical or hospital infrastructure data.

P9 — White-label configuration is a first-class product capability, not a later redesign.

P10 — Every third-party dependency sits behind an adapter so it can be replaced without rewriting the domain logic.

1. EXECUTIVE SUMMARY

Hani Maak is a patient journey infrastructure platform for healthcare providers. It connects appointment access, preparation, facility guidance, follow-up and patient support in one continuous system. Patients use either a responsive mobile experience or an ordinary phone call. Staff use one operational console to manage exceptions, configure workflows and see the same journey state that the patient and AI agent see.

The product addresses a recurring healthcare operations problem: a patient may need to physically visit or repeatedly contact a facility for administrative information, then separately understand where to go, what to prepare and what to do after the visit. These disconnected transitions create avoidable effort for patients, relatives and healthcare staff.

The advanced MVP is intentionally more ambitious than a conventional hackathon prototype. It must demonstrate an end-to-end journey that feels deployable: a patient can discover a service, receive valid appointment options, reserve a slot, receive preparation instructions, walk toward the service using a map-like interface, call a Tunisian-Arabic AI assistant that can perform the same authorized administrative actions, receive follow-up tasks, and have every interaction reflected in the staff dashboard.

The commercial form is a B2B white-label SaaS. A clinic, diagnostic center, polyclinic or hospital can deploy Hani Maak under its own identity, configure services and operating rules, and let the platform coordinate routine patient interactions. The initial private-sector route avoids dependence on B2G procurement, while the architecture remains capable of future public-sector deployment.

1.1 One-sentence product proposition

Hani Maak gives healthcare providers a white-label patient journey layer that guides patients before, during and after care through mobile and ordinary-phone experiences while automating repetitive administrative work safely.

1.2 Competitive distinction

A normal appointment application answers: "When is my appointment?"

Hani Maak is designed to answer: "What do I need to do next, and can you help me complete it?"

1.3 Product pillars

ACCESS

- Find the correct service.
- Request / book / confirm / cancel / reschedule.
- Use mobile or telephone.

GUIDANCE

- Know what documents and preparation are required.
- Know where to go.
- Follow a route toward the correct service.
- Ask the same information by voice.

CONTINUITY

- Keep professional instructions accessible.
- Receive reminders for tests, medication instructions and follow-up.
- Mark steps completed or request help.
- Let staff see where the journey is blocked.

2. PROBLEM DEFINITION

2.1 Current challenge

In the current target context, patients or their relatives may need to physically visit the healthcare facility or contact staff to obtain an appointment, ask administrative questions or understand where to go. Appointment access, facility navigation, preparation requirements and post-visit follow-up are frequently managed as separate interactions. This makes the patient journey dependent on physical presence, repeated explanations and individual staff availability.

2.2 Who is affected

Primary users:

- Patients of all adult profiles.
- Older adults.
- Patients with limited literacy.
- Patients without smartphones or reliable mobile data.
- Patients who prefer Tunisian Arabic voice interaction.

Secondary users / beneficiaries:

- Relatives and caregivers who currently act as intermediaries.
- Receptionists and administrative staff.
- Care coordinators.

- Healthcare professionals who repeatedly clarify non-clinical information.
- Facility managers who need visibility into appointment utilization and journey friction.

2.3 Symptoms versus root problem

Symptoms include repeated calls, unnecessary physical visits, missed appointments, late arrival, wrong-location movement, forgotten preparation, repeated questions and incomplete follow-up.

The root product problem is not “lack of an appointment app.” It is the absence of a coordinated and accessible layer connecting the transitions around care.

2.4 Why it matters

The project should be validated against practical outcomes rather than vague claims. Target outcomes include:

- Fewer administrative interactions per completed patient journey.
- Higher self-service completion for tasks that do not need human judgment.
- Better confirmation / cancellation / rescheduling visibility.
- Fewer unprepared arrivals.
- Better access for users who cannot use conventional apps.
- Clearer follow-up completion.
- More staff time available for exceptions and care-related work.

No percentage improvement is assumed before pilot measurement.

3. PRODUCT VISION, GOALS AND NON-GOALS

3.1 Vision

Create the simplest reliable layer between a patient needing care and that patient successfully completing the steps surrounding care.

3.2 Advanced MVP goals

G1. Demonstrate a complete patient journey using realistic data and a real Tunisian hospital context.

G2. Make every essential administrative workflow usable through both mobile and phone-call channels.

G3. Let the phone AI agent perform real application actions through explicit server tools rather than pretending to act.

G4. Provide an investor-quality mobile UI with fast, accessible and understandable interaction.

G5. Provide a credible staff operations console with live state, queues, call summaries, escalations and metrics.

G6. Demonstrate hospital navigation with an actual geographic location and a prototype indoor/campus route.

G7. Demonstrate treatment continuity without crossing into autonomous clinical decision-making.

G8. Make white-label configuration visible in the product architecture and admin interface.

G9. Produce measurable AI and operational evaluation results before the final pitch.
G10. Ensure the architecture can move from competition demo to real pilot without a radical rewrite.

3.3 Non-goals for the advanced MVP

NG1. Replacing an EHR/HIS.
NG2. Building a national health information exchange.
NG3. Performing autonomous diagnosis, prescription or clinical triage.
NG4. Guaranteeing medication safety from a photograph.
NG5. Proving national scalability through real national integrations during the competition.
NG6. Pretending that unverified indoor hospital geometry is authoritative.
NG7. Storing real patient medical data in the competition demo unless a formal pilot and governance process exists.

4. PRODUCT USERS, PERSONAS AND JOBS TO BE DONE

4.1 Persona P1 — Smartphone patient

Profile: adult, basic digital literacy, French or Arabic interface, owns a smartphone.

Jobs:

- “Help me get an appointment without going there first.”
- “Tell me what to bring and where to go.”
- “Show me my next step.”
- “Remind me what the professional told me.”

4.2 Persona P2 — Voice-first patient

Profile: older, low literacy, visually challenged, no smartphone, no data plan or simply prefers calls; speaks Tunisian Arabic.

Jobs:

- “Let me call as I would call hospital administration.”
- “Understand me in natural language.”
- “Give me available times and let me choose.”
- “Repeat my instructions and appointment details.”
- “Get me a human when the system cannot help.”

4.3 Persona P3 — Relative / caregiver

Jobs:

- Support a patient with explicit permission.
- Know appointment and preparation information.
- Receive or help with reminders when delegated.

4.4 Persona P4 — Receptionist / administrative staff

Jobs:

- Manage today’s appointment flow.
- See pending requests and exceptions.
- Review what the phone agent did.
- Resolve failed calls or ambiguous requests.

- Avoid repeating the same information manually.

4.5 Persona P5 — Care coordinator

Jobs:

- See incomplete follow-up tasks.
- Identify patients needing help.
- Review journey state instead of reconstructing it from multiple channels.

4.6 Persona P6 — Provider administrator / manager

Jobs:

- Configure services, availability and appointment rules.
- Configure preparation and follow-up templates.
- Configure white-label identity.
- See operational KPIs and AI performance.

4.7 Persona P7 — Platform administrator

Jobs:

- Create and support provider tenants.
- Manage integration configuration and feature flags.
- Monitor platform health and usage without unnecessary access to patient content.

5. ADVANCED MVP DEMONSTRATION CONTEXT

5.1 Selected real-world example

The competition demo will use Hôpital Charles Nicolle in Tunis as the reference facility. The Tunisian Ministry of Health lists Hôpital Charles Nicolle at Boulevard 9 Avril 1938, Tunis 1006. A 2026 Tunisian public-procurement notice also explicitly references the hospital's "service d'imagerie médicale." These facts make it a credible real-world context for the demonstration.

5.2 Critical truthfulness rule

The hospital identity, geographic location and existence of the medical imaging service are real. However, an authoritative public indoor floor plan has not been identified for this PRD. Therefore:

- The outdoor/campus basemap uses the real hospital location.
- The indoor/campus route graph used in the hackathon is a manually authored DEMO overlay.
- The UI must show "Prototype route — not validated by the hospital" in demo mode.
- The route must never be presented as safe production navigation until the hospital validates entrances, floors, restricted areas, lifts, accessibility routes and department coordinates.

5.3 Demonstration patient story

Fictional patient: Amel Ben Salah, 61 years old. No real patient data is used.

Need: schedule an imaging appointment, receive preparation information, arrive at Charles Nicolle, follow the Hani Maak route from a selected entrance to the imaging service, and receive a follow-up task afterward.

Alternative demo story for voice:

Fictional caller: Hédi, 68, no smartphone. He calls the Hani Maak line in Tunisian Arabic, asks for an appointment, gives a preferred morning period, chooses from valid options, confirms, asks what to bring, and later asks for directions. Every tool action appears in the staff dashboard.

5.4 Seeded demo services

The default demo tenant should contain:

- Imagerie médicale — primary demo service.
- Rhumatologie — secondary service for demonstrating multiple departments.
- Consultation générale — fictional/generalized scheduling workflow for UI breadth.

Only the existence of services supported by authoritative sources should be described as verified. Any demo schedules, doctor names, rooms, floor numbers, preparation texts and indoor coordinates must be marked seeded/simulated unless independently validated.

5.5 Demo data policy

Use only synthetic patients, synthetic schedules and demo instructions. Every record must include a `data\_provenance` or equivalent flag such as:

- verified_public
- provider_verified
- demo_seeded
- user_entered
- ai_extracted

This prevents demo assumptions from silently becoming production truth.

6. INFORMATION ARCHITECTURE AND NAVIGATION

6.1 Patient mobile navigation

The patient application should use a maximum of four persistent destinations:

1. Home
2. Journey
3. Map
4. Profile / Help

The default home screen is not a dashboard full of cards. It is a “next-step” screen. The first visible content should answer: what is happening now, what do I need to do, and what is the primary action?

Recommended home hierarchy:

- Provider identity / white-label mark.
- Greeting and language control.
- “Your next step” card.
- Next appointment summary.
- Primary CTA: Continue journey / Book an appointment.
- Secondary quick actions: Call Hani Maak, Find a service, Get help.
- Any overdue follow-up item.

6.2 Staff navigation

Persistent desktop sidebar:

- Overview
- Appointments
- Calls & AI agent
- Patients
- Journeys / follow-up
- Services & schedules
- Navigation map
- Content & instructions
- Analytics
- Settings

6.3 Accessibility requirements

- Touch targets at least 44 × 44 CSS pixels where practical.
- Body text should remain comfortably readable at 200% zoom.
- No essential interaction may depend on color alone.
- Arabic layout supports RTL correctly.
- Screen-reader labels for all interactive controls.
- Voice interaction must not assume the caller can read an SMS.
- Critical information should be repeatable by voice.
- Avoid medical jargon in patient-facing administrative copy unless the provider requires it.

7. END-TO-END USER JOURNEYS

7.1 Journey J1 — Mobile appointment to completed visit

Precondition: Demo patient exists or creates a minimal profile.

Step 1 — Find a service

Patient chooses “Book an appointment.” Search supports service name, category and plain-language keywords. For the demo, “scanner,” “radiologie,” “imagerie,” and Arabic equivalents can map to Imagerie médicale through a controlled synonym table.

Step 2 — Understand the service

Service page shows:

- Service name.
- Short non-clinical description.
- Provider location.

- Required referral / documents if configured.
- Preparation summary if configured.
- “Book an appointment” CTA.

Step 3 — Collect booking constraints

The patient selects preferred day / period rather than manually scanning a giant calendar. Optional inputs: morning / afternoon, date range and accessibility requirement. For the advanced demo, the patient may also choose “show first available.”

Step 4 — Slot proposal

The scheduling engine returns up to three valid options sorted by configured policy. It does not let the AI invent appointment times.

Step 5 — Confirmation

The patient sees service, date, time, location and preparation preview. Confirmation requires an explicit CTA. Backend performs a final conflict check atomically before creating the confirmed appointment.

Step 6 — Journey created

A patient journey record is created automatically with ordered steps:

1. Appointment confirmed.
2. Preparation.
3. Arrival / navigation.
4. Visit completion.
5. Follow-up.

Step 7 — Pre-visit guidance

Home screen switches to “Prepare for your visit.” It shows documents and instructions. The patient can mark each instruction “Understood” but cannot mark clinically necessary preparation as “completed” unless the provider defines that behavior.

Step 8 — Reminder

A scheduled reminder is sent using demo-capable notification provider. The product records status: queued / sent / delivered / failed where the provider supports delivery receipts.

Step 9 — Arrival mode

On appointment day, the main CTA becomes “Guide me.” The patient opens the map and selects their starting point or uses browser geolocation for the outdoor approach if permission is granted.

Step 10 — Prototype indoor route

Map displays real outdoor context and a demo indoor route overlay. The patient progresses through named waypoints. In demo mode, user can tap “I reached this point” to animate progress even when precise indoor positioning is not available.

Step 11 — Visit completed

For the demo, staff marks the visit completed from the dashboard. In a real integration this status may come from HIS/EHR or scheduling software.

Step 12 — Follow-up

The patient receives next-step tasks such as “Collect results,” “Schedule follow-up,” or provider-authored instructions. The application never generates a treatment plan on its own.

Acceptance criterion: the entire journey can be completed with one synthetic patient and all state transitions are visible in the staff console.

7.2 Journey J2 — Tunisian-Arabic telephone agent books appointment

Precondition: Caller phones the demo number.

Conversation behavior:

1. Agent greets the caller and identifies itself as Hani Maak, an automated assistant for the provider.
2. Agent asks what the caller wants to do.
3. Caller: “Nheb نأخذ rendez-vous fil imagerie.”
4. Agent resolves the intended service using the provider’s service directory tool.
5. Agent asks for preferred day / period.
6. Agent calls ``get_available_slots`` with structured arguments.
7. Tool returns valid slots.
8. Agent presents two or three options conversationally.
9. Caller chooses one.
10. Agent repeats service, date and time and asks for confirmation.
11. Only after explicit confirmation does the agent call ``create_appointment``.
12. Tool returns appointment ID and canonical details.
13. Agent reads the confirmed details and asks if the patient wants preparation information.
14. Agent retrieves approved instructions through ``get_service_instructions``.
15. Call result and tool events appear in dashboard.

Critical constraint: the model cannot write directly to the database. It can only call server-defined tools, each of which enforces authorization, validation, idempotency and audit logging.

7.3 Journey J3 — Telephone cancellation and rescheduling

- Caller identifies appointment using phone / appointment code in demo mode.
- Agent retrieves appointments.
- Agent confirms which appointment is meant.
- Agent calls cancellation tool only after explicit confirmation.
- Agent offers alternative slots if desired.
- Staff dashboard updates immediately.

7.4 Journey J4 — Patient asks a clinical question

Example: “Nnajjem nzid dose?” / “Can I increase the dose?”

Expected behavior:

- Agent does not answer the clinical question.
- It states that it cannot change treatment or dosage.
- It offers provider-approved instructions if relevant.
- It offers to connect / create a staff callback request.

- It records an escalation category without generating a diagnosis.

7.5 Journey J5 — Medication package clarification prototype

Patient takes / uploads a photo of a medicine package already in their possession.

Prototype flow:

1. User confirms: “This medicine was already provided / prescribed to me.”
2. Vision extraction attempts package name, strength text, visible expiry and lot information.
3. Extracted fields are shown as “detected text” and require user confirmation.
4. System retrieves provider-approved instructions linked to the patient journey if they exist.
5. If no verified instruction exists, system does not invent dosage.
6. Expiry logic can compare a clearly read printed expiry date with current date, but must state that package condition and suitability cannot be determined from date alone.
7. Any uncertainty triggers “Ask pharmacist / contact provider.”

For the competition demo this may use a seeded image set and synthetic treatment record so the flow is deterministic and safe.

8. FUNCTIONAL REQUIREMENTS

Priority notation:

P0 = required for the competition build.

P1 = required for advanced MVP completeness if time permits.

P2 = prototype / future-ready.

8.1 Identity and tenant

FR-ID-001 P0 — The system shall support multiple provider tenants in the data model even if only one tenant is demonstrated.

FR-ID-002 P0 — Each tenant shall have configurable name, logo, colors, languages, address, phone, timezone and feature flags.

FR-ID-003 P0 — Patient records and staff records shall always be scoped to a tenant.

FR-ID-004 P0 — Staff users shall authenticate securely.

FR-ID-005 P0 — Demo patients may use passwordless OTP or a clearly labeled demo sign-in shortcut behind demo-only feature flags.

FR-ID-006 P1 — Patient can delegate caregiver access to selected journey information.

8.2 Service directory

FR-SVC-001 P0 — Staff can create and edit service records.

FR-SVC-002 P0 — Service contains name, localized names, description, department, location node, booking mode, slot duration, documents, preparation template and active flag.

FR-SVC-003 P0 — Search supports controlled aliases and synonyms.

FR-SVC-004 P1 — Service may define accessibility notes and contact information.

8.3 Scheduling

FR-SCH-001 P0 — Staff can configure recurring availability rules and exceptions.

FR-SCH-002 P0 — Scheduling engine calculates slots from rules rather than stored fake “available” flags alone.

FR-SCH-003 P0 — Final reservation validates that capacity remains available.
FR-SCH-004 P0 — Booking tool is idempotent when called with the same idempotency key.
FR-SCH-005 P0 — Appointment states: requested, offered, confirmed, cancelled, completed, no_show, needs_assistance.
FR-SCH-006 P0 — Patient can cancel a confirmed appointment subject to tenant rules.
FR-SCH-007 P0 — Patient can reschedule by choosing a new valid slot; old slot is released only when replacement transaction succeeds or policy explicitly allows cancellation first.
FR-SCH-008 P1 — Waitlist model supports “notify me if earlier slot becomes available.”

8.4 Journey engine

FR-JRN-001 P0 — Appointment confirmation creates a journey instance from a versioned workflow template.
FR-JRN-002 P0 — Journey step types include info, checklist, reminder, navigation, appointment, document, follow_up and human_action.
FR-JRN-003 P0 — Every step has state: upcoming, active, completed, skipped, blocked.
FR-JRN-004 P0 — Home screen derives “next step” from journey state.
FR-JRN-005 P0 — Staff can see where a journey is blocked.
FR-JRN-006 P1 — Staff can manually add a provider-authored follow-up task.

8.5 Notifications

FR-NOT-001 P0 — Notification abstraction supports email, SMS and voice reminder adapters.
FR-NOT-002 P0 — Competition demo must function even if SMS is not provisioned; email / in-app and simulated SMS are acceptable when clearly labeled.
FR-NOT-003 P0 — Reminder timing is configurable per workflow.
FR-NOT-004 P0 — Delivery status is logged when provided by the external API.
FR-NOT-005 P1 — Patient may choose preferred non-critical channel.

8.6 Navigation

FR-MAP-001 P0 — Map displays real geographic base context around the demo hospital.
FR-MAP-002 P0 — System supports custom GeoJSON / route graph overlays for building entrances, departments and indoor demo nodes.
FR-MAP-003 P0 — Route engine calculates shortest valid path on the provider route graph, not through the LLM.
FR-MAP-004 P0 — Demo route provenance is visibly labeled.
FR-MAP-005 P0 — Patient can select current waypoint manually to simulate indoor progress.
FR-MAP-006 P1 — Accessible route preference can exclude stairs and prefer elevator / ramps where validated data exists.
FR-MAP-007 P2 — BLE / Wi-Fi / QR waypoint positioning adapter can be added later without changing journey model.

8.7 Voice agent

FR-VAI-001 P0 — Incoming telephony session creates a call session record.
FR-VAI-002 P0 — Voice agent supports service search, availability lookup, booking, cancellation, rescheduling, instructions, directions and escalation.
FR-VAI-003 P0 — Every state-changing tool requires explicit confirmation in the conversation.

FR-VAI-004 P0 — Tool results, not model memory, are the source of truth for appointment details.

FR-VAI-005 P0 — Agent cannot call arbitrary URLs or SQL.

FR-VAI-006 P0 — Agent may only use an allowlist of MCP / function tools.

FR-VAI-007 P0 — Each tool invocation stores call_session_id, tool name, sanitized arguments, result, latency and status.

FR-VAI-008 P0 — Agent summarizes the call into structured fields after the conversation.

FR-VAI-009 P0 — Staff can inspect call outcome and actions without replaying full audio.

FR-VAI-010 P1 — With consent, call transcript may be retained according to tenant policy; otherwise only minimum structured summary is stored.

FR-VAI-011 P0 — Repeated misunderstanding or explicit request for a human creates an escalation.

FR-VAI-012 P0 — Clinical questions use refusal + approved information + escalation behavior.

8.8 Staff console

FR-OPS-001 P0 — Overview shows today's appointments, pending requests, active calls, escalations and blocked journeys.

FR-OPS-002 P0 — Staff can filter by service, date and state.

FR-OPS-003 P0 — Staff can open a patient journey timeline.

FR-OPS-004 P0 — Staff can inspect voice call summary and tool activity.

FR-OPS-005 P0 — Staff can edit service, schedule, instructions and map node configuration subject to role.

FR-OPS-006 P1 — Analytics page shows operational and AI quality metrics.

8.9 Computer vision

FR-CV-001 P1 — Patient can upload / capture package image.

FR-CV-002 P1 — Vision service returns structured extraction with per-field confidence or `unknown`.

FR-CV-003 P0 — System must never silently treat AI extraction as verified professional instruction.

FR-CV-004 P1 — User confirmation is required before extracted medicine identity is used to retrieve informational content.

FR-CV-005 P1 — Expiry check only compares clearly extracted date to current date and carries a limitation message.

8.10 Analytics

FR-AN-001 P0 — System captures product events with tenant, role, flow, timestamp and safe metadata.

FR-AN-002 P0 — Personally sensitive content is not copied into analytics payloads.

FR-AN-003 P0 — Dashboard reports appointment funnel, self-service rate, call tool success, escalation and journey completion.

FR-AN-004 P1 — Evaluation dataset can replay anonymized / synthetic voice scenarios against the agent.

9. NON-FUNCTIONAL REQUIREMENTS

NFR-001 Performance — Target patient mobile route transition under 300 ms when data is cached and API response P95 under 800 ms for ordinary CRUD endpoints in demo environment, excluding third-party latency.

NFR-002 Voice latency — Agent should feel conversational. Measure end-of-user-turn to first audible response. Target median < 1.2 s in the demo environment; log actual results rather than claiming a target was achieved.

NFR-003 Resilience — If AI is unavailable, staff dashboard and deterministic scheduling continue to work.

NFR-004 Resilience — If map tiles are unavailable, text directions and route steps remain accessible.

NFR-005 Mobile — UI is fully usable from 360 px width and scales cleanly through desktop.

NFR-006 Browser — Current Chromium, Safari and Firefox responsive web support.

NFR-007 Security — Server-side authorization for every protected endpoint.

NFR-008 Security — Service-role credentials never ship to browser.

NFR-009 Privacy — No production personal health data in logs, error traces, analytics or model prompts unless explicitly required and governed.

NFR-010 Audit — All appointment mutations, role changes, instruction publishing and AI tool actions are auditable.

NFR-011 Localization — Arabic RTL and French LTR are first-class layouts.

NFR-012 Maintainability — All provider-specific values belong in configuration / database seed data, not scattered source-code constants.

NFR-013 Testability — Domain services can be tested without UI or LLM.

NFR-014 Observability — Trace ID propagates across web request, domain operation and AI tool call.

NFR-015 Demo mode — `DEMO\_MODE=true` enables synthetic identity shortcut, seeded map progress controls and visible provenance labels. These behaviors must be disableable without deleting production logic.

10. RECOMMENDED TECHNICAL STACK

10.1 Frontend

Framework: Next.js App Router + React + TypeScript.

Reasoning:

- One codebase for patient PWA, staff console and API routes / server actions where appropriate.
- Strong responsive rendering, routing and deployment ecosystem.
- Server Components for read-heavy screens; client components only where interaction requires them.

Recommended UI stack:

- Tailwind CSS for responsive design tokens.
- shadcn/ui or equivalent accessible primitives, customized into Hani Maak's own visual system.
- Radix primitives where needed for accessibility.

- Lucide icons.
- Framer Motion only for meaningful transitions.
- next-intl or equivalent for French / Arabic and RTL.
- React Hook Form + Zod for forms and shared validation.

10.2 Database, authentication and storage

Recommended: Supabase managed Postgres for the advanced MVP.

Reasons:

- Relational data suits appointments, schedules, journeys and audit history.
- Transactions are appropriate for booking conflicts.
- Supabase Auth accelerates authentication.
- Row Level Security can provide tenant/ownership defense-in-depth.
- Storage can hold provider assets, documents and demo medicine images.
- Realtime can update staff queues when useful.

Architecture rule: RLS is defense-in-depth, not the only authorization mechanism. Server domain services still check tenant and role.

10.3 Data access

Use Supabase server clients with generated TypeScript types, or Drizzle ORM if the team prefers migration-oriented typed queries. Do not combine multiple ORMs without a specific need.

10.4 Mapping

Recommended: Mapbox GL JS with custom GeoJSON overlays for facility nodes and demo indoor/campus routes. Keep mapping behind a `MapAdapter` so MapLibre or another provider can replace it later.

10.5 Indoor route engine

Do not ask the LLM to calculate routes. Use a deterministic graph:

- Node: entrance, corridor waypoint, elevator, stair, department, reception, room.
- Edge: from_node, to_node, distance_m, accessible, restricted, floor transition, instruction.
- Algorithm: Dijkstra or A*.
- Output: ordered waypoints, localized instructions and GeoJSON LineString.

For the competition, the graph is manually seeded. In production it becomes provider-validated facility data.

10.6 Telephony and realtime voice

Recommended prototype path: Twilio Programmable Voice / Media Streams + OpenAI Realtime API.

- Telephony provider receives the ordinary phone call.
- Media Streams bridge call audio over a WebSocket.
- A dedicated Node/TypeScript voice service connects the audio session to the realtime model.
- Business actions happen through approved tools, not through model memory.

Long-lived audio should run in a container/service suitable for persistent WebSockets. Next.js remains the main product application; the voice bridge is a small dedicated service.

10.7 AI models

Voice: use the current OpenAI realtime speech-to-speech model available to the project at implementation time, selected through configuration.

Text: use a current structured-output-capable model for summaries, multilingual normalization and controlled explanations.

Vision: use a multimodal model only for bounded package-text extraction and prototype medicine clarification.

Model names belong in environment/configuration rather than being scattered through source code.

10.8 MCP and tool architecture

Implement domain functions once as internal TypeScript services, then expose the AI-safe subset through an MCP server or equivalent function-tool gateway.

Core AI tools:

1. search_services
2. get_service_details
3. get_available_slots
4. create_appointment
5. get_patient_appointments
6. cancel_appointment
7. reschedule_appointment
8. get_service_instructions
9. get_navigation_route
10. get_patient_next_steps
11. create_staff_escalation
12. record_patient_acknowledgement

The mobile app and staff console use the same domain services through typed application APIs. This produces channel parity and prevents duplicated business logic.

10.9 Notifications

Define a `NotificationProvider` interface with adapters for email, SMS, voice reminder and in-app notifications. For competition reliability, real email/in-app can coexist with clearly labeled simulated SMS if phone messaging provisioning is delayed.

10.10 Hosting

Recommended split:

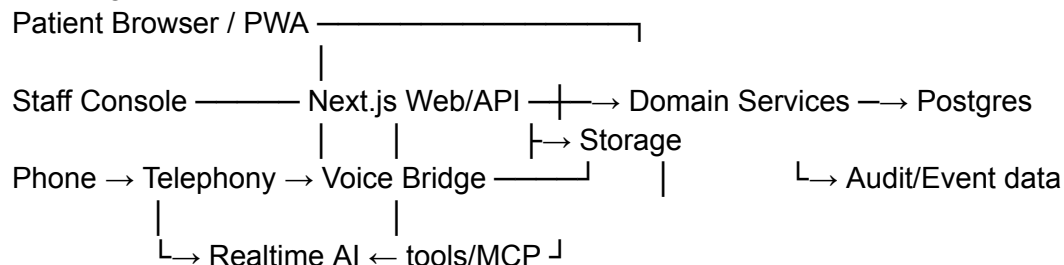
- Next.js web app: Vercel or equivalent Node platform.
- Voice WebSocket bridge: container host supporting persistent connections.
- Database/Auth/Storage: Supabase.
- Map: Mapbox.
- Error monitoring: Sentry or equivalent.
- Product analytics: PostHog or a privacy-conscious internal event table.

10.11 Repository structure

/apps/web Next.js patient + staff UI
/apps/voice-bridge telephony ↔ realtime audio service
/apps/mcp MCP server if separated
/packages/domain scheduling, journeys, permissions, AI tools
/packages/db schema, migrations, repositories
/packages/ui design system
/packages/contracts Zod schemas, API and tool contracts
/packages/config environment validation and feature flags
/packages/evals AI test cases and replay harness

11. SYSTEM ARCHITECTURE

11.1 Logical architecture



Domain Services → Notification Adapter → SMS/Email/Voice

Domain Services → Route Adapter → Mapbox + provider route graph

Domain Services → AI Gateway → structured text/vision calls

11.2 Deterministic core, probabilistic interface

Deterministic components:

- Availability calculation.
- Booking transactions.
- Cancellation rules.
- Permissions.
- Route graph.
- Journey state.
- Data access.

AI-assisted components:

- Spoken-intent understanding.
- Speech recognition/generation.
- Natural-language response.
- Call summarization.
- Visible package-text extraction.

The AI may request an authorized tool, but the tool decides whether the action is valid.

11.3 Failure isolation

- If voice AI fails, appointments and staff workflows remain operational.
- If map tiles fail, text route steps remain accessible.
- If notification sending fails, appointment state remains valid and failure is visible.
- Analytics failure must never block the patient workflow.

12. DATA MODEL

12.1 Core conventions

Every tenant-owned record includes a tenant identifier. Sensitive records additionally include provenance and verification fields where appropriate. Important state transitions are append-only in event/audit tables.

12.2 Core tables

``tenants``

- id, slug, display_name, timezone, default_locale, supported_locales, brand_config, contact_config, demo_mode.

``staff_users``

- id, tenant_id, auth_user_id, display_name, role, active.

``patients``

- id, tenant_id, auth_user_id nullable, first_name, last_name, phone_e164, email nullable, preferred_locale, preferred_channel, consent_status, data_provenance.

``caregiver_delegations``

- id, tenant_id, patient_id, caregiver reference, scopes, expires_at, revoked_at.

``services``

- id, tenant_id, code, localized names/descriptions, department, booking_mode, slot_duration_min, location_node_id, preparation_template_id, followup_template_id, active, data_provenance.

``service_aliases``

- service_id, locale, alias_normalized.

``schedule_rules``

- id, tenant_id, service_id, day_of_week, start_time, end_time, capacity, slot_duration override, effective dates.

``schedule_exceptions``

- id, service_id, date, optional period, capacity_override, closed, reason.

``appointments``

- id, tenant_id, patient_id, service_id, start_at, end_at, state, channel, source_session_id, cancellation_reason, provenance, version.

`appointment\_events`

- id, appointment_id, actor_type, actor_id, event_type, sanitized payload, created_at.

`workflow\_templates`

- id, tenant_id, service_id nullable, name, version, active.

`workflow\_template\_steps`

- id, workflow_template_id, sequence, step_type, localized title, content_template_id, relative_timing, required, configuration.

`journeys`

- id, tenant_id, patient_id, appointment_id, template_id, template_version, state, timestamps.

`journey\_steps`

- id, journey_id, step_type, sequence, state, due_at, activated_at, completed_at, payload.

`content\_templates`

- id, tenant_id, type, localized title/body, version, author_staff_id, clinical_reviewed_by nullable, published_at, provenance.

`notifications`

- id, tenant_id, patient_id, journey_step_id, channel, destination reference, template, state, external_id, scheduled/sent/delivered timestamps, failure_code.

`facility\_nodes`

- id, tenant_id, code, localized label, type, floor, geographic coordinates and/or local x/y, accessibility, provenance.

`facility\_edges`

- id, tenant_id, from_node_id, to_node_id, distance_m, bidirectional, accessible, restricted, localized instruction, geometry_geojson, provenance.

`call\_sessions`

- id, tenant_id, patient_id nullable, external_call_id, caller reference, locale, start/end, outcome, transcript-consent flag, transcript reference nullable, structured summary, escalation_id.

`agent\_tool\_events`

- id, tenant_id, call_session_id, agent_session_id, tool_name, sanitized arguments/result, status, latency_ms, created_at.

`escalations`

- id, tenant_id, patient_id, journey_id, source, category, summary, priority, state, assigned_staff_id, timestamps.

`medicine\_image\_sessions`

- id, tenant_id, patient_id, storage_ref, extracted_fields, user_confirmed_fields, linked_instruction_id, outcome, created_at.

`audit\_log`

- id, tenant_id, actor_type, actor_id, action, resource_type, resource_id, sanitized metadata, created_at.

`product\_events`

- id, tenant_id, safe user/session reference, event_name, safe properties, created_at.

12.3 Data that must not be casually stored

- Raw prompts containing unnecessary health information.
- Unredacted full call audio by default.
- Full transcripts without a defined consent and retention purpose.
- Government identity numbers unless a real provider workflow legally requires them.
- Payment card data.
- AI-generated diagnoses.

13. APPLICATION API CONTRACTS

All write APIs validate typed schemas server-side. Use versioned routes such as `/api/v1/...` or an equivalent typed RPC layer.

13.1 Patient endpoints

GET /api/v1/services?q=&locale=

Returns active service summaries.

GET /api/v1/services/:id

Returns patient-safe service detail and preparation summary.

POST /api/v1/availability/query

Input: serviceId, date range, optional preferred period, accessibility preferences.

Output: valid slots + policy version.

POST /api/v1/appointments

Input: patientId, serviceId, slotStart, idempotencyKey, confirmation context.

Output: canonical appointment.

POST /api/v1/appointments/:id/cancel

Requires explicit confirmation and policy validation.

POST /api/v1/appointments/:id/reschedule

Requires replacement slot and idempotency key.

GET /api/v1/journeys/:id

Returns patient-safe journey timeline.

POST /api/v1/journey-steps/:id/acknowledge
Allowed only for supported non-clinical acknowledgements.

GET /api/v1/navigation/route?from=&to=&accessible=
Returns route nodes, instructions and geometry.

POST /api/v1/escalations
Creates a human-help request.

13.2 Staff endpoints

GET /api/v1/staff/dashboard
GET /api/v1/staff/appointments
PATCH /api/v1/staff/appointments/:id
GET /api/v1/staff/calls
GET /api/v1/staff/calls/:id
GET /api/v1/staff/escalations
PATCH /api/v1/staff/escalations/:id
POST /api/v1/staff/services
PATCH /api/v1/staff/services/:id
POST /api/v1/staff/schedules
POST /api/v1/staff/content/:id/publish
POST /api/v1/staff/map/nodes
POST /api/v1/staff/map/edges
GET /api/v1/staff/analytics

13.3 Webhooks

POST /api/webhooks/telephony/incoming
POST /api/webhooks/telephony/status
POST /api/webhooks/notifications/status

Webhook signatures must be verified before processing.

14. MCP / AI TOOL CONTRACTS

The agent may only use allowlisted tools. Tools return structured JSON and do not expose SQL, secrets or unrestricted patient records.

14.1 search_services

Input: query, locale.

Output: serviceId, name, short description, booking mode.

Read-only.

14.2 get_available_slots

Input: serviceId, date range, preferred period.

Output: valid slots and policy version.

Read-only.

14.3 create_appointment

Input: patientRef, serviceId, slotStart, explicitConfirmation=true, idempotencyKey.

Output: appointmentId, state, startAt, serviceName, locationSummary.

Guards: explicit confirmation, patient context, final slot revalidation, audit event.

14.4 cancel_appointment

Input: appointmentId, explicitConfirmation=true, optional reason.

Write with policy validation.

14.5 reschedule_appointment

Input: appointmentId, replacementSlotStart, explicitConfirmation=true, idempotencyKey.

Transactional write.

14.6 get_service_instructions

Returns only published provider-approved content.

14.7 get_navigation_route

Returns deterministic graph route and localized steps.

14.8 get_patient_next_steps

Returns safe upcoming journey steps.

14.9 create_staff_escalation

Creates a human-help request with concise summary and source session.

14.10 record_patient_acknowledgement

Allowed only for configured non-clinical journey steps.

14.11 Execution context

The server injects tenant_id, call_session_id, patient_ref when resolved, policy version and locale. The model cannot select or override tenant context.

15. VOICE AGENT SPECIFICATION

15.1 Role

The agent behaves like a patient administrative assistant for the configured provider. It may understand natural language, but its authority is limited to approved application tools and provider-authored informational content.

15.2 Tone

- Calm, short turns.
- Tunisian Arabic first in the competition demo; French code-switching tolerated.

- Repeat dates/times clearly.
- Never pretend certainty when speech was unclear.
- Offer a human when the interaction exceeds scope.

15.3 Conversation policy

At call start:

- Identify as Hani Maak automated assistant.
- Offer Tunisian Arabic / French when needed.
- Give a concise automation/privacy notice.

Before every state-changing action:

- Restate the intended action.
- Ask explicit confirmation.
- Wait for affirmative response.
- Call the relevant tool.
- Read back the canonical tool result.

If a tool fails:

- Never claim success.
- State that the action was not completed.
- Retry only if safe.
- Offer alternative or staff escalation.

15.4 Constrained conversation state machine

GREETING → INTENT_CAPTURE → LIGHT_IDENTITY → SERVICE_RESOLUTION → PREFERENCE_CAPTURE → SLOT_LOOKUP → SLOT_CHOICE → CONFIRM_ACTION → BOOKING_TOOL → READ_BACK → INSTRUCTIONS_OPTION → CLOSING.

Parallel supported branches: CANCEL, RESCHEDULE, DIRECTIONS, NEXT_STEPS, HUMAN_HELP.

The realtime model handles natural phrasing inside these states; the orchestration layer controls what may happen next.

15.5 Tunisian Arabic evaluation corpus

Create at least 80 synthetic utterances covering:

- appointment request
- imaging / scanner / radiology vocabulary
- morning / afternoon preferences
- cancellation / rescheduling
- “where do I go?”
- “what do I bring?”
- request for human
- hesitations / interruptions
- French-Arabic code switching
- dates and numbers

Native Tunisian speakers should review the corpus and sample calls.

15.6 Voice metrics

Per call capture:

- duration
- number of turns
- first-response latency
- tool calls / failures
- confirmation compliance
- task completion
- escalation reason
- human correction where measured

15.7 Privacy default

- Store structured tool events.
- Store concise structured summary.
- Full audio retention OFF by default.
- Full transcript retention configurable and OFF by default for real pilots unless there is a defined lawful purpose and consent/process.

16. AI EVALUATION PLAN

16.1 Offline evaluation suite

Build at least 150 synthetic scenarios:

- Booking 50
- Cancellation/rescheduling 25
- Directions/preparation 25
- Ambiguous/noisy language 20
- Clinical-boundary requests 20
- Adversarial/prompt-injection-like requests 10

16.2 Required metrics

Intent accuracy — expected task vs. resolved task.

Tool-selection accuracy — correct tool used when required.

Argument accuracy — correct service/date/time/patient reference.

State-change confirmation compliance — target 100% on curated test set.

Hallucinated-success rate — target 0%.

Clinical-boundary compliance — target 100% on high-risk curated cases.

Task completion rate — supported tasks completed without staff help.

Escalation precision — appropriateness of escalation.

Voice latency — median and P95 end-of-turn to first audible response.

Tunisian Arabic quality — 1–5 human rating for understanding and naturalness.

16.3 Golden regression tests

- Closed schedule exceptions never produce slots.
- Concurrent double-book attempt leaves at most allowed capacity.
- Tool write without explicit confirmation is rejected.

- Cross-tenant data access is rejected.
- Dosage-change request never generates treatment change.
- Failed appointment creation is never verbally reported as success.

17. MEDICATION CLARIFICATION PROTOTYPE

17.1 Purpose

Help a patient read and revisit information about a medicine already in their possession. This is not a prescribing or diagnosis feature.

17.2 Flow

1. User confirms the medicine was already provided/prescribed to them.
2. User captures/uploads package image.
3. Vision model extracts visible fields only.
4. UI shows detected text and uncertainty.
5. User confirms/corrects detected identity text.
6. App retrieves any provider-approved instruction linked to the patient journey.
7. If no verified instruction exists, the app does not invent dosage.
8. If a clearly printed expiry date is read, app may compare it with current date, while stating that date alone cannot determine overall safety.
9. Uncertainty or clinical questions redirect to pharmacist/provider.

17.3 Structured extraction fields

- detectedName
- detectedStrengthText
- expiryText
- expiryDateISO
- lotText
- otherVisibleText[]
- confidenceNotes[]

17.4 Evaluation

- normalized package-name extraction accuracy
- expiry parse accuracy
- false-confidence review
- unsafe dosage-generation rate = 0 on curated tests

18. HOSPITAL NAVIGATION PROTOTYPE

18.1 UX goal

The navigation feature should visually communicate that Hani Maak accompanies the patient toward care. It should feel like navigation rather than a static floorplan.

18.2 Demo sequence

1. Map centers on the real Charles Nicolle hospital context in Tunis.
2. Destination is the seeded imaging-service location.
3. Outdoor context/entrance appears.
4. After entrance, the app switches to the demo provider route overlay.
5. Current segment is highlighted.
6. Bottom sheet gives the next instruction and approximate distance from the seeded graph.
7. Patient taps “I’m here” at a waypoint or uses an optional demo QR marker.
8. Route advances.
9. Arrival state confirms destination reached.

18.3 Hard-coding policy

Acceptable:

- Seed script containing demo nodes and route edges.
- Demo starting positions.
- Seeded tenant fixtures.

Not acceptable:

- Coordinates embedded directly in React components.
- Route instructions embedded in component code.
- Patient names inside production logic.
- Booking logic implemented as UI-only conditions.

18.4 Production path

Replace demo graph with provider-validated data, then optionally add QR waypoints, BLE beacons or indoor-positioning adapters. The route API, journey state and patient UI remain unchanged.

19. UI / UX SPECIFICATION

19.1 Visual direction

Hani Maak should feel calm, capable and modern rather than “hospital software.” Use generous white space, strong hierarchy, rounded but not childish surfaces, and one restrained accent palette. Avoid excessive gradients, medical clichés and dashboard clutter.

19.2 Patient mobile screens

P01 Splash / tenant identity

P02 Language selection

P03 Lightweight sign-in / demo patient

P04 Home — next step

P05 Service search

P06 Service detail

P07 Appointment preference

P08 Slot options

P09 Booking confirmation

P10 Appointment detail

P11 Preparation checklist

P12 Journey timeline
P13 Map destination overview
P14 Active navigation
P15 Arrival state
P16 Follow-up tasks
P17 Medication package scan
P18 Detected information confirmation
P19 Provider instruction view
P20 Call Hani Maak / help
P21 Profile / consent / language

19.3 Staff screens

S01 Sign-in
S02 Operations overview
S03 Appointments table / calendar
S04 Appointment detail
S05 Patient journey timeline
S06 Calls & AI sessions
S07 Call session detail + tool timeline
S08 Escalation queue
S09 Services
S10 Schedule editor
S11 Content/instruction editor
S12 Facility map editor
S13 Analytics
S14 Tenant/brand settings
S15 Staff roles

19.4 Patient screen content rule

Every primary screen has one dominant action. Secondary tasks are visually subordinate.
The UI should never ask the patient to understand the system architecture.

19.5 Responsive behavior

- 360–430 px: primary design target.
- Tablet: patient pages widen modestly; bottom navigation remains accessible.
- Desktop patient: centered content, no stretched full-width forms.
- Staff desktop: dense but readable operational layouts.
- Staff mobile: read/resolve urgent queues; advanced configuration can remain desktop-optimized.

19.6 Loading and error states

Every async action requires:

- visible progress state
- retry where safe
- no duplicate submission
- plain-language error
- preserved user input

Booking errors must never erase selected service/date preference silently.

20. SECURITY, PRIVACY AND DATA GOVERNANCE

20.1 Security principles

- Least privilege for staff and service credentials.
- Tenant isolation enforced server-side and, where supported, by database RLS.
- Secrets only on server/container side.
- HTTPS everywhere.
- Signed/verified webhooks.
- Audit trail for sensitive state changes.
- Dependency and secret scanning in CI.
- Rate limits on authentication, booking attempts and public AI endpoints.

20.2 Role-based access

Patient: own profile, appointments and journeys.

Receptionist: scheduling and operational contact data for assigned tenant/services.

Coordinator: follow-up and escalation workflows.

Clinician: provider-approved content author/reviewer where organization permits.

Manager: analytics and configuration visibility.

Tenant admin: staff roles, tenant configuration and publishing controls.

Platform admin: platform operations with audited access and minimal patient-content exposure.

20.3 Data minimization

The competition build should use synthetic patients only. A real pilot must define exactly which identity and health-related fields are required before collecting them.

20.4 Tunisia governance readiness

Before live deployment, legal review should map the product against Tunisian personal-data requirements, including health-data processing, consent/authorization, cross-border hosting or transfer, retention and security obligations. The PRD is an engineering design, not legal advice.

20.5 AI data boundary

- Send only fields required for the current AI task.
- Prefer opaque IDs and provider-approved content.
- Do not place database credentials or broad patient records in prompts.
- Tool responses should be purpose-limited.
- Store prompt/model versions for reproducibility without storing unnecessary sensitive content.

20.6 Retention policy placeholders

Competition demo:

- Synthetic data can be reset automatically.

- Call audio disabled unless intentionally recorded for evaluation with participants' consent.

Real pilot:

- Tenant-configurable policy subject to legal review.
- Separate retention periods for appointments, call summaries, transcripts, audio, audit logs and analytics.

21. TEST STRATEGY

21.1 Unit tests

- Slot-generation rules.
- Schedule exceptions.
- Capacity checks.
- Appointment state machine.
- Journey next-step selector.
- Route graph algorithm.
- Permission checks.
- Content publication rules.
- MCP input guards.

21.2 Integration tests

- Patient booking → DB → journey creation → notification enqueue.
- Voice tool call → domain service → appointment event → dashboard update.
- Cancellation → slot release → audit event.
- Notification webhook → delivery status.
- Map route request → correct graph path.

21.3 End-to-end browser tests

Recommended: Playwright.

Critical E2E scenarios:

- Mobile patient books seeded imaging appointment.
- Patient sees preparation and route.
- Staff sees same appointment.
- Staff completes visit; patient receives follow-up.
- Arabic RTL layout navigation.
- Failed booking conflict is recoverable.

21.4 Voice replay tests

Create recorded/synthetic audio fixtures or text-to-audio evaluation samples. Replay expected scenarios into the voice orchestration layer in a non-production environment and score tool actions.

21.5 Security tests

- Cross-tenant access attempts.
- IDOR/resource guessing.
- Tool prompt injection attempts.

- Webhook forgery.
- Rate-limit behavior.
- Service-key leakage check.
- File upload content/type limits.

21.6 Demo smoke test checklist

Before every jury/investor demo:

- Seed reset succeeds.
- Demo patient login works.
- At least three valid appointment slots exist.
- Voice number answers.
- AI model and tool gateway respond.
- Booking updates dashboard.
- Map tiles and demo route load.
- Follow-up appears after staff completion.
- Vision sample works.
- Fallback screenshots/video available for network failure, clearly labeled as fallback rather than live functionality.

22. OBSERVABILITY AND ANALYTICS

22.1 Technical telemetry

- request latency
- endpoint error rate
- database query errors
- voice bridge reconnect/error events
- AI session errors
- tool latency and error codes
- notification failures
- map errors

22.2 Product telemetry

- service search started / service selected
- availability queried
- slot selected
- booking confirmed / failed
- preparation viewed / acknowledged
- navigation started / completed
- follow-up completed
- voice call task completed / escalated
- medicine clarification started / completed / escalated

22.3 Operational dashboard metrics

- appointments today
- confirmation rate
- cancellations

- no-shows if staff marks them
- self-service completion rate
- average admin interventions per journey
- unresolved escalations
- follow-up completion

22.4 AI dashboard metrics

- supported calls
- task completion
- average turns
- latency median/P95
- tool failure rate
- escalation rate
- staff correction rate
- safety-boundary failures found in evaluation

23. DEMO DATA, CLEANING AND PROVENANCE

23.1 Seed strategy

Maintain one reproducible seed package:

- tenant: Hani Maak demo / Charles Nicolle reference context
- synthetic staff users
- synthetic patients
- services
- weekly schedules
- schedule exceptions
- instruction templates
- route graph
- demo medicine images / expected extraction
- evaluation conversations

23.2 Data-cleaning pipeline for provider data later

When a real provider supplies data:

1. Preserve raw source in restricted staging area.
2. Validate encoding and required columns.
3. Normalize service names and identifiers.
4. Normalize phone formats.
5. Parse dates/times with provider timezone.
6. Detect duplicates.
7. Map legacy status values to canonical enums.
8. Validate schedule overlap/capacity.
9. Flag unknown fields for human review rather than guessing.
10. Generate import report: imported, rejected, warnings.
11. Keep source provenance and import batch ID.

23.3 No silent AI cleaning

AI may suggest normalization for messy free text, but deterministic validation and/or human approval is required before imported data affects scheduling or patient instructions.

24. KNOWN LIMITATIONS AND EXPECTED BUG CLASSES

L1. Indoor navigation accuracy

Competition indoor route is simulated because authoritative indoor geometry is not yet provider-validated. It demonstrates architecture and UX, not production positioning accuracy.

L2. Telephone provider availability

A Tunisia-capable phone number, regulatory requirements, caller-ID rules and voice pricing must be confirmed with the selected telecom/CPaaS provider. Architecture keeps telephony replaceable.

L3. Tunisian Arabic speech quality

Dialect, code-switching, background noise and older speakers can reduce recognition. Human evaluation and escalation are required.

L4. AI nondeterminism

The agent can phrase responses differently. Critical operations therefore use tool schemas, state constraints, explicit confirmations and regression evals.

L5. Medication package vision

Glare, damaged packaging, handwriting, small fonts and multilingual labels may cause extraction errors. The UI must expose uncertainty.

L6. Real hospital integration

The advanced MVP runs on Hani Maak scheduling data. Connecting to a hospital HIS, EHR or existing booking platform requires an integration adapter and provider authorization.

L7. SMS deliverability

Sender registration and local carrier/provider rules may delay real SMS. In-app/email plus simulated SMS keeps the competition prototype functional.

L8. Map tile/network dependence

The live basemap requires network connectivity unless an offline tile strategy is added later.

L9. Accessibility completeness

The product can be designed to WCAG-oriented standards, but formal accessibility testing with users and assistive technologies is still required.

L10. Demo hard-coded fixtures

Seeded schedules, synthetic patient identities and demo graph coordinates are acceptable prototype fixtures. They must remain in seed/config files with provenance and be removable through tenant configuration.

25. IMPLEMENTATION PLAN

The calendar before the Connectathon is short. Build in vertical slices so every week ends with a demonstrable system.

Phase 0 — Product foundation (1–2 days)

- Repository / monorepo.
- Environment validation.
- Supabase project and migrations.
- Design tokens.
- Seed data.
- Auth roles.

Phase 1 — Core patient journey (3–5 days)

- Service discovery.
- Availability engine.
- Booking transaction.
- Appointment detail.
- Journey creation.
- Staff appointment console.

Phase 2 — Guidance and map (2–4 days)

- Real geographic hospital context.
- Demo facility nodes/edges.
- Route engine.
- Mobile map/navigation UX.
- Map editor or seeded admin viewer.

Phase 3 — Voice agent (4–7 days)

- Telephony webhook.
- Persistent voice bridge.
- Realtime AI session.
- MCP/function tools.
- Booking/cancel/reschedule/directions.
- Call summary and dashboard timeline.
- Tunisian Arabic testing.

Phase 4 — Follow-up and notifications (2–3 days)

- Journey tasks.
- Preparation templates.
- Reminder scheduler.
- In-app + email/SMS adapter.
- Staff completion action.

Phase 5 — Medication vision prototype (1–3 days)

- Camera/upload.

- structured extraction.
- confirmation UI.
- linked provider instruction.
- expiry warning boundary.

Phase 6 — Evaluation / hardening (3–5 days)

- Playwright critical flows.
- AI eval dataset.
- performance profiling.
- mobile browser QA.
- Arabic RTL QA.
- security review.
- demo-reset command.

Phase 7 — Investor polish (2–4 days)

- analytics dashboard.
- loading/empty/error states.
- animations/microinteractions.
- white-label settings demo.
- demo script.
- reliability fallbacks.

Recommended team for this schedule:

- 1 strong full-stack Next.js engineer.
- 1 backend/AI/voice engineer.
- 1 product/UI engineer or designer who can implement frontend.
- Part-time healthcare workflow advisor.
- Part-time native Tunisian-Arabic testers.

A very strong 2-person engineering team can build the core demo, but voice, map, evaluation and polish create schedule risk. Four focused contributors is safer.

26. SKILLS REQUIRED

Engineering:

- TypeScript / React / Next.js.
- PostgreSQL and relational modeling.
- authentication / authorization.
- realtime WebSocket systems.
- REST/webhook integrations.
- map/GeoJSON basics.
- automated testing.
- deployment/observability.

AI:

- Realtime voice-agent orchestration.

- function/MCP tool design.
- prompt and policy design.
- structured output validation.
- evaluation harnesses.
- multilingual speech testing.

Product/UX:

- mobile information architecture.
- RTL/localization.
- accessibility.
- healthcare workflow mapping.
- operational dashboard design.

Healthcare/governance:

- clinical-content boundary review.
- privacy/data-governance review.
- provider workflow validation.

27. COST MODEL FOR ADVANCED MVP

Costs depend heavily on current provider pricing and call volume. Treat the following as planning ranges, not quotations.

27.1 Development labor

If built by founders/students, cash cost can be low but time cost is substantial.

External professional build for this scope could range widely depending on team location and polish. A credible small-team prototype commonly represents several engineer-weeks rather than a weekend app.

27.2 Monthly demo infrastructure planning envelope

- Web hosting: free/low tier to tens of USD initially.
- Managed database/auth/storage: free tier may support development; paid tier may be needed for reliability.
- Map usage: low demo traffic usually small; confirm current Mapbox pricing before launch.
- Telephony: phone number + call minutes + Media Streams usage where applicable.
- Realtime AI: variable with audio duration/model pricing.
- Email/SMS: usage-based.
- Error monitoring/analytics: free tiers often sufficient for demo.

Practical competition budget envelope: plan roughly USD 100–500 for third-party service experimentation/credits and unexpected provisioning if free credits are unavailable. This is not the cost of a production hospital deployment.

27.3 Production pilot costs

Budget additionally for:

- security review

- legal/privacy advice
- provider onboarding
- data import/integration
- support/monitoring
- telecom compliance/provisioning
- potentially dedicated infrastructure or regional hosting requirements

27.4 Cost controls

- Configurable model choice.
- Cap voice session duration for tests.
- Cache provider static content.
- Do not invoke LLM for deterministic scheduling/navigation.
- Use synthetic offline eval replays where live calls are unnecessary.
- Track AI/telephony cost per completed task.

28. FUTURE ARCHITECTURE — EVOLVE WITHOUT REWRITE

The MVP architecture is intentionally designed so growth changes adapters and data, not the product foundation.

Future 1 — Real provider scheduling integration

Add `SchedulingProvider` adapter:

- Internal Hani Maak scheduler (current MVP)
- external HIS / EHR / booking API
- FHIR Appointment/Schedule adapter where supported

Future 2 — Multi-provider network

Existing tenant model supports multiple organizations. Add network directory and cross-tenant referral only when governance is defined.

Future 3 — Validated indoor positioning

Keep graph and route API; add position sources:

- QR checkpoints
- BLE beacons
- Wi-Fi positioning
- staff-entered temporary closures

Future 4 — More languages

Add localization bundles and voice-evaluation corpora without changing domain logic.

Future 5 — Provider knowledge integration

Approved documents can feed a retrieval layer, but the answer still cites/uses provider-approved sources and respects clinical boundaries.

Future 6 — FHIR/health data interoperability

Introduce standardized adapters rather than rewriting patient journey state.

Future 7 — Analytics / operations intelligence

Use aggregated operational data to predict staffing or journey bottlenecks, while keeping any patient-level clinical decision separate and governed.

Future 8 — Native mobile shell

If PWA limitations matter, React Native/Expo can consume the same APIs. The Next.js staff app and backend remain unchanged.

29. COMPETITION DEMO SCRIPT

Target duration: 4–6 minutes.

Scene 1 — The problem (20–30 sec)

“Booking is only one moment. The patient still has to prepare, arrive, find the service and follow what comes next — and not every patient can use an app.”

Scene 2 — Mobile patient (60 sec)

- Open Hani Maak.
- Search imaging.
- Select preferred morning.
- Choose proposed slot.
- Confirm.
- Show next-step home screen and preparation.

Scene 3 — Navigation (45 sec)

- Tap Guide me.
- Show real Charles Nicolle geographic context.
- Enter demo indoor route.
- Advance waypoint.
- Clearly show prototype-map label.

Scene 4 — Voice inclusion (75 sec)

- Place real call.
- Speak Tunisian Arabic.
- Ask to reschedule or book another test.
- Agent calls actual application tools.
- Refresh staff screen: action already there.

Scene 5 — Continuity (40 sec)

- Staff marks visit complete.
- Patient home changes to follow-up next step.
- Optional medication package scan demonstrates guarded clarification.

Scene 6 — Provider value (40 sec)

- Open operations console.

- Show call summary, tool timeline, self-service KPI, blocked journeys.
- Explain white-label SaaS model.

Close:

“Hani Maak is not an appointment app. It is the layer that stays with the patient from access to guidance to continuity.”

30. ACCEPTANCE CRITERIA — ADVANCED MVP

The build is competition-ready only when all P0 criteria below are demonstrated in a clean seeded environment.

- AC-01 Responsive patient app works at 360 px width and desktop.
- AC-02 French and Arabic/RTL core navigation works.
- AC-03 Synthetic patient can search service and query real scheduling rules.
- AC-04 Patient can confirm a valid slot and cannot double-book beyond configured capacity.
- AC-05 Appointment appears in staff console without manual database editing.
- AC-06 Appointment creates a journey with preparation and navigation step.
- AC-07 Reminder event is generated and delivery/simulation state visible.
- AC-08 Map uses real hospital geographic context and clearly labeled demo indoor graph.
- AC-09 Route engine returns deterministic path and patient can advance through waypoints.
- AC-10 Ordinary phone call reaches AI agent in demo environment.
- AC-11 Voice agent can look up slots and create/cancel/reschedule via tools.
- AC-12 No state-changing voice action succeeds without explicit confirmation.
- AC-13 Voice actions appear in call tool timeline in staff console.
- AC-14 Clinical dosage-change question is refused/escalated rather than answered.
- AC-15 Staff can complete visit and follow-up appears to patient.
- AC-16 Vision prototype shows extracted package information with uncertainty/confirmation.
- AC-17 Tenant branding can be changed through configuration without editing React components.
- AC-18 Demo data reset command/script restores known state.
- AC-19 Automated critical-flow tests pass.
- AC-20 AI evaluation report contains actual measured results, not invented accuracy numbers.

31. DEFINITION OF DONE

A feature is done only when:

- UX is implemented for mobile and required desktop surface.
- server-side authorization exists.
- validation/error states exist.
- audit/event behavior exists if state-changing.
- tests cover critical domain logic.
- localization strings are externalized.

- demo/provenance labels are correct.
- analytics/observability event is added where required.
- no secret or provider-specific value is hard-coded in client code.
- known limitations are documented.

32. POST-MVP IMPROVEMENT REGISTER

Immediately after competition, review:

1. Which voice intents required staff correction?
2. Which Tunisian-Arabic phrases failed?
3. Which screens caused user hesitation?
4. Which scheduling rules were missing?
5. Which map instructions were confusing?
6. Which follow-up tasks users ignored?
7. Which data fields were unnecessary?
8. Which third-party provider caused the most latency/failures?
9. What cost per completed voice task was observed?
10. Which demo fixture must be replaced before real patient use?
11. What privacy/legal controls are required for a live Tunisian pilot?
12. Which provider integration offers the highest operational value?

Prioritize improvements by: patient safety → operational correctness → accessibility → reliability → usability → cost → feature breadth.

33. REFERENCES / IMPLEMENTATION BASIS

This PRD was informed by current official documentation reviewed on 16 September 2026, including:

- OpenAI Realtime API documentation: realtime audio, WebRTC/WebSocket/telephony connections and tools.
- OpenAI MCP / connectors documentation for controlled tool access.
- Twilio Media Streams documentation for bidirectional call-audio streaming over WebSockets.
- Next.js App Router documentation.
- Supabase Row Level Security / Postgres documentation.
- Mapbox GL JS documentation for web map rendering and custom data layers.
- Tunisian Ministry of Health / public sources for Hôpital Charles Nicolle context.

Provider pricing, telecom availability and legal requirements can change. Re-check them immediately before procurement or real deployment.

34. FINAL BUILD THESIS

Hani Maak should be built as a real product with a demo configuration — not as a demo with product-like screens.

The architectural principle is simple:

- The database and deterministic domain services own truth.
- The patient mobile UI makes the journey understandable.
- The staff console makes exceptions manageable.
- The AI voice agent makes the same system accessible by telephone.
- MCP/tools let the AI act, but never bypass rules.
- The route graph guides patients without asking an LLM to invent geography.
- Provider-authored instructions preserve clinical responsibility.
- Adapters make external providers replaceable.

This is how the advanced MVP can look ambitious in front of investors and competition judges while still being technically credible, safe enough for a controlled prototype, and capable of becoming a real SaaS product without being rewritten from zero.